

Quiz 03 – Dynamic Arrays and Amortized Analysis: The Snake Challenge

Estructuras de Datos 2026-2 – Universidad Nacional de Colombia

Nombre: _____ Documento: _____ Grupo: _____

1. Pregunta 1 – Dynamic Array

Representación. Un arreglo nativo `data` (creado como `(T[]) new Object[n]`), `size` y `capacity`. El elemento lógico i vive en `data[i]`. Se valida `initialCapacity ≥ 1` (`IllegalArgumentException`), índices en $[0, size)$ (`IndexOutOfBoundsException`) y `removeLast` vacío (`NoSuchElementException`). Si `size = capacity`, `append` llama `resize(2*capacity)`, que copia elemento por elemento.

b) Invariantes (antes y después de cada operación pública): **I1** $1 \leq capacity$ y $0 \leq size \leq capacity$; **I2** `capacity = data.length`; **I3** para $0 \leq i < size$, `data[i]` es el i -ésimo elemento lógico (prefijo contiguo, en orden); **I4** `data[i]=null` para $size \leq i < capacity$; **I5** `capacity` nunca disminuye y vale `initialCapacity · 2r` tras r `resizes`.

c) Complejidad worst-case. `get`, `set`: $O(1)$ (una verificación y un acceso directo `data[i]`). `removeLast`: $O(1)$ (decremento, lectura y `null`; no hay ciclo ni contracción). `append`: $\Theta(n)$ en el peor caso, porque cuando `size = capacity = n` el `resize` ejecuta un ciclo de n copias; sin `resize` es $O(1)$.

d) El worst-case acota una operación; el costo amortizado acota el promedio sobre *cualquier secuencia* de operaciones. No hay contradicción porque la operación cara es rara: tras un `resize` a $2C$ hacen falta al menos C `append` baratos antes del siguiente. En n `appends` desde capacidad 1 se copian $1 + 2 + \dots + 2^{\lfloor \log_2(n-1) \rfloor} < 2n$ elementos, así que el costo total es $< 3n$ y el promedio es $O(1)$, aunque un `append` individual cueste $\Theta(n)$.

2. Pregunta 2 – Representación con corrimiento

a) En `MOVE` se ejecuta primero `append`, así que al llamar `REMOVE_TAIL` hay $k+1$ elementos y el ciclo hace `size - 1 = k` asignaciones: $\Theta(k)$ (en general $\Theta(size)$), en todos los casos, no solo en el peor.

b) Con longitud constante k cada movimiento cuesta $O(1)$ (`append` sin `resize`) + $\Theta(k)$ (corrimiento). Solo el primer movimiento puede provocar un `resize` ($O(k)$), porque después la capacidad ya es $\geq k+1$ y nunca se reduce. Total: $T(m, k) = \Theta(mk) + O(k) = \Theta(mk)$.

c) La afirmación es falsa. El costo amortizado de `MOVE` es $T(m)/m$ y $T(m) = \sum_{j=1}^m (c_j^{\text{app}} + c_j^{\text{rem}}) \geq \sum_j c_j^{\text{rem}} = mk$. Luego $T(m)/m \geq k$, que no es $O(1)$ porque k no está acotada. Que `append` sea $O(1)$ amortizado solo acota la primera suma; no dice nada de la segunda. El análisis amortizado reparte el costo de operaciones caras *raras*, pero aquí *todas* las llamadas a `REMOVE_TAIL` cuestan k : no hay nada que repartir. `MOVE` es $\Theta(k)$ amortizado (solo sería $O(1)$ si k fuera constante).

d) Bottleneck: la representación fija la cola en la posición física 0. Borrar el primer elemento de un bloque contiguo anclado al inicio obliga a mover los k restantes. El problema no es mover datos, sino tener fijo el *inicio*: la solución es mover el índice de inicio en lugar de los datos.

3. Pregunta 3 – Rediseño: buffer circular

a) Estado. `Position[] data` (único arreglo, gestionado por `Snake`); `tail`: índice físico del elemento lógico 0 (la cola); `size`: número de elementos k ; `capacity = data.length`. La cabeza no se almacena: está en `phys(size - 1)`. Solo hay 3 enteros adicionales, un número constante. (`resizeCount` y `copyCount` son instrumentación para verificar la P4, no forman parte del diseño.)

b) Función lógico→físico. $\text{phys}(i) = (\text{tail} + i) \bmod \text{capacity}$, $0 \leq i < \text{size}$.

Siempre es un índice válido: $0 \leq \text{tail} < \text{capacity}$ y $0 \leq i < \text{size} \leq \text{capacity}$, luego $0 \leq \text{tail} + i < 2 \text{capacity}$ y el residuo de un entero no negativo está en $[0, \text{capacity})$ (el `%` de Java coincide con `mod` para operandos no negativos). Como $2 \text{capacity} \leq 2^{31} - 1$ por el control de `nextCapacity`, no hay overflow. Además, $i \mapsto (\text{tail} + i) \bmod C$ es inyectiva en $[0, C)$, así que elementos lógicos distintos nunca comparten celda.

c) Operaciones (código en `Snake.java`). `addHead(p)`: si `size = capacity` hace `resize(2C)`; luego `data[phys(size)]=p`, `size++`. `removeTail()`: lee `data[tail]`, la pone en `null`, hace `tail=(tail+1)%C`, `size--`. Son $O(1)$ operaciones sin ciclos: $O(1)$ *worst-case*. `get(i)`: valida $0 \leq i < \text{size}$ y retorna `data[phys(i)]`: $O(1)$.

d) resize(C'). Crea un arreglo nuevo y copia `newData[i]=data[phys(i)]` para $i = 0, \dots, \text{size} - 1$ (con la capacidad *vieja*); luego `tail=0`, `capacity=C'`. Así se “desenrolla” el círculo: el elemento lógico i queda en la posición física i sin importar cómo estaba repartido antes. Cuesta $\Theta(\text{size})$.

e) Invariantes y corrección. **J1** $1 \leq \text{capacity} = \text{data.length}$, $0 \leq \text{size} \leq \text{capacity}$; **J2** $0 \leq \text{tail} < \text{capacity}$; **J3** para $0 \leq i < \text{size}$: `data[phys(i)]=pi` (orden cola→cabeza); **J4** las celdas `phys(size), ..., phys(capacity - 1)` son `null`; **J5** `capacity` nunca disminuye.

addHead: tras el posible `resize`, `size < C`, así que por **J4** la celda `phys(size)` está libre; escribir allí p e incrementar `size` hace que p sea el nuevo p_k (cabeza) y **J3** se mantiene. *removeTail:* retorna `data[tail]=p0`. Con `tail' = tail+1`: $\text{phys}'(i) = (\text{tail} + 1 + i) \bmod C = \text{phys}(i + 1)$, es decir, el nuevo elemento i es el antiguo $i + 1$: **J3** se mantiene y la celda liberada pasa a ser la última libre (**J4**). *resize:* `newData[i] = pi` y `tail = 0` implican $\text{phys}'(i) = i$: **J3** y **J4**

se cumplen. Conocer solo **size** y **capacity** *no* basta para reconstruir el arreglo: cualquier rotación $\text{tail} \in [0, C)$ es posible; **tail** es justamente la información extra.

f) Traza (java Main; “_” = celda vacía; head = phys(size − 1) es derivado).

Mov.	Arreglo físico	tail	head	size	cap.	resize	Orden lógico (cola→cabeza)
Inicial	[A B]	0	1	2	2	—	A, B
N1 no come	[_ B N1 _]	1	2	2	4	sí (2 copias)	B, N1
N2 come	[_ B N1 N2]	1	3	3	4	no	B, N1, N2
N3 no come	[N3 _ N1 N2]	2	0	3	4	no (da la vuelta)	N1, N2, N3
N4 no come	[N3 N4 _ N2]	3	1	3	4	no	N2, N3, N4
N5 come	[N3 N4 N5 N2]	3	2	4	4	no	N2, N3, N4, N5
N6 no come	[_ N3 N4 N5 N6 _ _ _]	1	4	4	8	sí (4 copias)	N3, N4, N5, N6

En N1, **addHead** ocurre antes que **removeTail** con $\text{size} = \text{capacity} = 2$: la serpiente necesita transitoriamente 3 celdas y se redimensiona aunque no crezca. En N6 el **resize** desenrolla $[N3, N4, N5, N2]$ ($\text{tail}=3$) a $[N2, N3, N4, N5]$ ($\text{tail}=0$) antes de escribir N6. Ningún movimiento ordinario desplaza elementos.

4. Pregunta 4 – Análisis amortizado

a) Aggregate Method. Sea L_t la longitud tras t movimientos: $L_0 = 2$, $L_t = L_{t-1} + [\text{come en } t]$, luego $L_t \leq 2 + g$ y L es no decreciente. Durante el movimiento t se almacenan como máximo $L_{t-1} + 1$ elementos (justo después de **addHead**). Por tanto

$$N_{\max} = \max_{1 \leq t \leq m} (L_{t-1} + 1) = L_{m-1} + 1 \leq g + 3$$

($= g + 3$ si el último movimiento no come; $= g + 2$ si come). Un **resize** desde capacidad C ocurre si y solo si en algún momento hay que guardar $C + 1$ elementos, es decir, si y solo si $C < N_{\max}$, y copia exactamente C elementos. Con capacidades $2, 4, 8, \dots$ y C_r la mayor potencia de 2 menor que $N_{\max}$:

$$\text{Copias} = 2 + 4 + \dots + C_r = 2C_r - 2 < 2N_{\max} - 2 \leq 2g + 4.$$

Así, $\text{Copias} = O(g + 1)$; para $g \geq 1$ es $\Theta(g)$, ya que también $\text{Copias} \geq C_r \geq N_{\max}/2 > g/2$. Con $g = 0$ hay exactamente un **resize** (2 copias, el del primer movimiento). Traza: $g = 2$, $N_{\max} = 5$, copias = $2 + 4 = 6 \leq 8$.

b) Por qué la suma geométrica es la correcta. Cada término es la capacidad existente en el instante del **resize** por tres hechos: (1) el **resize** solo se dispara con $\text{size} = \text{capacity}$, así que se copian exactamente C elementos, ni más ni menos; (2) la capacidad solo se duplica y nunca baja (**removeTail** no contrae), así que los valores C son distintos y crecientes, $C_{j+1} = 2C_j$, y ninguno se repite; (3) después de pasar a $2C$ con C elementos copiados, para volver a llenarse la serpiente debe crecer netamente C elementos más, lo que requiere al menos C nuevas escrituras. Léida desde el último **resize**, la suma es $C_r + C_r/2 + C_r/4 + \dots \leq 2C_r$: cada **resize** cuesta el doble del anterior pero está precedido por tantas escrituras nuevas como copió el anterior. Para almacenar N elementos hay a lo sumo N escrituras y menos de $2N$ copias: costo total $< 3N$, lineal en N .

c) Costo total. Cada movimiento hace una escritura (1), aritmética de índices $O(1)$ y, si no come, un **removeTail** $O(1)$. Luego

$$T(m) \leq cm + (2g + 4) \leq (c + 2)m + 4 = O(m) \quad (g \leq m).$$

El costo real de todas las llamadas a **addHead** es $m + \text{Copias} \leq m + 2g + 4 \leq 3m + 4$, así que su costo amortizado es $\leq 3 + 4/m = O(1)$, y más finamente $1 + O(\frac{g+1}{m})$.

d) Papeles de m y g . m determina el término $\Theta(m)$ del *movimiento*: una escritura y $O(1)$ de índices por paso, coma o no. g determina el término $O(g + 1)$ del *crecimiento*: la capacidad final y las copias dependen solo de la longitud máxima $N_{\max} \leq g + 3$. Los movimientos sin comer oscilan entre k y $k + 1$ y, como la capacidad no se reduce, a lo sumo uno por nivel de capacidad provoca **resize**; eso ya está contado en $N_{\max}$. Ejemplo: con $g = 0$ y $m = 10^6$ se hacen solo 2 copias (verificado en **Main**).

e) Banker’s Method. Se cobran 3 créditos por **addHead**: 1 paga la escritura y 2 se depositan en un *banco global* de la estructura (no sobre elementos individuales). **removeTail** se cobra 1, que paga su costo $O(1)$, y no toca el banco. *Afirmación:* al ocurrir un **resize** desde capacidad C el banco tiene $\geq C$ créditos; se pagan las C copias y el saldo nunca es negativo. *Base:* el estado inicial se construye con dos **addHead** en capacidad 2 (4 créditos depositados) y el primer **resize** copia 2. *Paso:* tras el **resize** anterior ($C/2 \rightarrow C$) quedaron $C/2$ elementos. Para que la estructura vuelva a estar llena ($\text{size} = C$) se necesita $\#\text{addHead} - \#\text{removeTail} = C/2$, luego $\#\text{addHead} \geq C/2$ y se depositaron $\geq C$ créditos. *Por qué removeTail no rompe el argumento:* los créditos están en el banco y no se pierden al borrar un elemento; además, cada eliminación *aumenta* el número de **addHead** necesarios para el próximo **resize**, lo que solo añade créditos. *Conclusión:* **addHead** ≤ 3 y **removeTail** ≤ 1 amortizados, total $\leq 4m + 2$. (*Alternativa con potencial:* $\Phi = 2 \text{size} - \text{capacity}$ sirve aquí porque en **MOVE** cada **removeTail** sigue a un **addHead**, así que $\text{size} \geq \text{capacity}/2$ y $\Phi \geq 0$. Para eliminaciones arbitrarias, el banco global es más robusto.)

5. Pregunta 5 – The Adversarial Snake

a) Sin contracción, la secuencia de capacidades $C_0 = 2$, $C_{j+1} = f(C_j)$ está fijada por la política. Por J5 y porque el **resize** solo ocurre con la estructura llena, el **resize** desde C_j sucede si y solo si la longitud máxima cumple $n > C_j$, y

siempre copia C_j . Entonces $\text{Copias}(n) = \sum_{C_j < n} C_j$ depende únicamente de n y de f : el adversario solo cambia *en qué movimiento* ocurre cada resize , no cuáles ocurren ni cuánto cuestan. Para aumentar las copias necesitaría que la capacidad pudiera *bajar* (una política de contracción) o poder *reducir la longitud* (`removeTail` sin `addHead`) junto con contracción. Así podría oscilar alrededor de un umbral y repetir resizes : con “duplicar al llenarse y reducir a la mitad si $\text{size} \leq C/2$ ”, alternar insertar/eliminar en la frontera cuesta $\Theta(n)$ por operación. La solución clásica es contraer solo cuando $\text{size} \leq C/4$.

b–e) Tabla ($C_0 = 2$, longitud máxima n ; espacio medido justo después de copiar, antes de escribir el elemento nuevo).

Política	# Resizes	Total copias	Costo amortizado por crecimiento	Espacio sin usar tras resize
$C + 1$	$n - 2 = \Theta(n)$	$\sum_{C=2}^{n-1} C = \frac{n(n-1)}{2} - 1 = \Theta(n^2)$	$\Theta(n)$	1 celda; $\frac{1}{C+1} \rightarrow 0$
$2C$	$\lceil \log_2 n \rceil - 1 = \Theta(\log n)$	$< 2n = \Theta(n)$	$\leq 3 = O(1)$	C celdas; $\frac{1}{2}$
$\lceil 1.5C \rceil$	$\approx \log_{1.5} \frac{n}{2} \approx 1.71 \log_2 n$	$< 3n = \Theta(n)$	$\leq 4 = O(1)$	$\approx C/2$ celdas; $\approx \frac{1}{3}$

Justificación. **A:** cada resize suma 1, se necesitan $n - 2$ para pasar de 2 a n y se copian $2, 3, \dots, n - 1$ elementos.

B: capacidades 2^j y la suma geométrica de la P4. **C:** $2, 3, 5, 8, 12, 18, \dots$; como $C_{j+1} \geq 1.5C_j$, se tiene $\sum C_j \leq C_r(1 + \frac{2}{3} + \frac{4}{9} + \dots) = 3C_r < 3n$. El costo amortizado por crecimiento es $(n + \text{copias})/n$. El espacio libre es $C_{\text{new}} - C$ y su fracción es $1 - C/C_{\text{new}}$.

f) Trade-off. Un factor mayor implica menos resizes y menos copias (menos tiempo), pero más memoria reservada sin usar. Un factor menor ahorra memoria a cambio de redimensionar más seguido y copiar más. El crecimiento aditivo ($C + 1$, o en general $C + c$) minimiza memoria pero da $\Theta(n^2/c)$ copias: el tiempo cuadrático lo hace inviable. Duplicar da amortizado 3 con hasta 50% libre; 1.5 da amortizado 4 con $\approx 33\%$ libre. Además, con 1.5 los bloques antiguos liberados pueden reutilizarse para un arreglo nuevo; con 2 nunca alcanzan.

Generalización $C_{\text{new}} = \lceil \alpha C \rceil$. Cualquier constante $\alpha > 1$ conserva costo amortizado constante. Con techo, $C_{j+1} \geq \alpha C_j$ y $C_{j+1} \geq C_j + 1$ (siempre crece). Si C_r es la última capacidad copiada, $C_r < n$ y $C_{r-i} \leq C_r/\alpha^i$:

$$\text{Copias}(n) = \sum_{i \geq 0} C_{r-i} \leq C_r \sum_{i \geq 0} \alpha^{-i} = C_r \frac{\alpha}{\alpha - 1} < \frac{\alpha}{\alpha - 1} n, \quad \text{Copias}(n) \geq C_r > \frac{n - 1}{\alpha}.$$

Para α constante, $\text{Copias} = \Theta(n)$ y $\approx \frac{n}{\alpha-1}$ a $\frac{\alpha n}{\alpha-1}$ según dónde caiga n . Además: costo amortizado $\leq 1 + \frac{\alpha}{\alpha-1} = \frac{2\alpha-1}{\alpha-1}$ (3 para $\alpha = 2$, 4 para 1.5, 6 para 1.25); $\#\text{resizes} = \Theta(\log_\alpha n)$; fracción libre $1 - \frac{1}{\alpha}$. Si $\alpha \rightarrow 1^+$ la constante $\frac{1}{\alpha-1}$ explota, y si $\alpha \leq 1$ no hay crecimiento. Con *piso* hace falta cuidado: $\lfloor 1.25 \cdot 2 \rfloor = 2$ y $\lfloor 1.25 \cdot 3 \rfloor = 3$, así que desde $C = 2$ la capacidad no crecería y la escritura se saldría del arreglo. Se corrige con $\max(C + 1, \lfloor \alpha C \rfloor)$, que es lo que el techo garantiza automáticamente.